

Guilherme Augusto NASCIMENTO

Informatique et Électronique pour les Systèmes Embarqués A4

Rapport Intermédiaire

Semaines 1 et 2

1 Modèles de Machine Synchrone

1.1 Premiers modèles

Pour la première recherche concernant les modèles de machines synchrones couramment utilisés pour les jumeaux numériques, le livre sur le contrôle et les systèmes d'énergie et de puissance a été révisé. Quelques concepts fondamentaux sur les machines synchrones, les systèmes d'énergie et leurs modèles ont été abordés. Pour effectuer des premiers tests de simulation et comparer ces modèles, une implémentation sur MATLAB a été réalisée, dont le but est de comprendre la résolution numérique des équations différentielles des modèles.

Les modèles mathématiques utilisent une série d'équations différentielles pour décrire le comportement des grandeurs qui représentent un processus ou un objet réel. [1] propose des modèles qui considèrent les forces électromotrices (f.e.m.) transitoires, subtransitoires ainsi que les équations d'oscillation. Une brève explication de chaque modèle est donnée ci-dessous, et la section suivante présente quelques graphiques obtenus lors de la simulation.

1.1.1 Modèle d'ordre 2

Le premier modèle testé est de deuxième ordre ; il considère les champs permanents et est décrit uniquement par les équations de 'swing' (Eq. 1 et Eq. 2).

$$\dot{\delta} = \Delta\omega \quad (1)$$

$$M\Delta\dot{\omega} = P_m - P_e - D\Delta\omega \quad (2)$$

1.1.2 Modèle d'ordre 3 et d'ordre 4

Les modèles d'ordre 3 ou 4 ne considèrent pas l'effet des enroulements amortisseurs. Ainsi, la machine est représentée uniquement par les f.e.m. transitoires E_q et E_d qui sont calculées par l'Eq. 3.

$$\begin{bmatrix} V_d \\ V_q \end{bmatrix} = \begin{bmatrix} E'_d \\ E'_q \end{bmatrix} - \begin{bmatrix} R & X'_q \\ X'_d & R \end{bmatrix} \begin{bmatrix} I_d \\ I_q \end{bmatrix} \quad (3)$$

Selon ces considérations, le modèle d'ordre 4 est décrit ci-dessous par les Eq. 4 et Eq. 5, ainsi que par les équations du modèle d'ordre 2 (Eq. 1 et Eq. 2).

$$T'_{do}\dot{E}'_q = E_f - E'_q + I_d(X_d - X'_d) \quad (4)$$

$$T'_{qo}\dot{E}'_d = -E'_d - I_q(X_q - X'_q) \quad (5)$$

Le modèle d'ordre 3 considère la f.e.m. transitoire de l'axe direct (axe-d) constante. De plus, s'il n'y a pas de bobines dans l'axe en quadrature (axe-q), il considère $E'_d = 0$ et $X'_q = X_q$, et est donc décrit par les Eq. 1, Eq. 2 et Eq. 4. La puissance électrique peut être calculée par les Eq. 6 (modèle d'ordre 4) et Eq. 7 (modèle d'ordre 3).

$$P_e = E'_q I_q + E'_d I_d + (X'_d - X'_q) I_d I_q \quad (6)$$

$$P_e = E'_q I_q + (X'_d - X'_q) I_d I_q \quad (7)$$

1.1.3 Modèle d'ordre 5 et d'ordre 6

Finalement, les modèles d'ordre 5 et 6 représentent la machine par les f.e.m. subtransitoires, selon l'Eq. ??.

$$\begin{bmatrix} V_d \\ V_q \end{bmatrix} = \begin{bmatrix} E''_d \\ E''_q \end{bmatrix} - \begin{bmatrix} R & X''_q \\ -X''_d & R \end{bmatrix} \begin{bmatrix} I_d \\ I_q \end{bmatrix} \quad (8)$$

Ces modèles considèrent l'effet des enroulements amortisseurs. Le modèle d'ordre 6 est défini par les Eq. 9 et Eq. 10 ci-dessous, ainsi que par les Eq. 4, Eq. 5, Eq. 1 et Eq. 2.

$$T''_{do}\dot{E}''_q = E'_q - E''_q + I_d(X'_d - X''_d) \quad (9)$$

$$T''_{qo}\dot{E}''_d = E'_d - E''_d + I_q(X'_q - X''_q) \quad (10)$$

Le modèle d'ordre 5, tout comme le modèle d'ordre 3, ne considère pas les variations de la f.e.m. transitoire E'_d ; par conséquent, l'Eq. 5 peut être négligée. La puissance électrique pour ces modèles est alors calculée par l'Eq. 11.

$$P_e = E'_d I_d + E'_q I_q + (X'_d - X'_q) I_d I_q \quad (11)$$

1.1.4 Simulation

Pour réaliser les premiers exemples de simulation et appréhender l'environnement MATLAB, les modèles ont été simulés et certains graphiques obtenus sont présentés ci-dessous. La simulation a été réalisée directement par la résolution des équations différentielles présentées, en utilisant la fonction `ode45` de MATLAB. L'implémentation du modèle d'ordre 6 est déposée dans l'Annexe A, tout comme un exemple d'utilisation de la fonction `ode45`. Pendant la simulation, deux perturbations ont été effectuées pour visualiser le comportement des modèles : un changement de la puissance mécanique de la turbine, suivi d'un court-circuit aux bornes de la machine (représenté par une chute de tension).

Les paramètres de la machine ont été tirés du livre comme exemple et sont listés avec les équations utilisées pour calculer les conditions initiales de chaque modèle dans l'Annexe A. La Fig. 1 montre le comportement décrit par le modèle d'ordre 2.

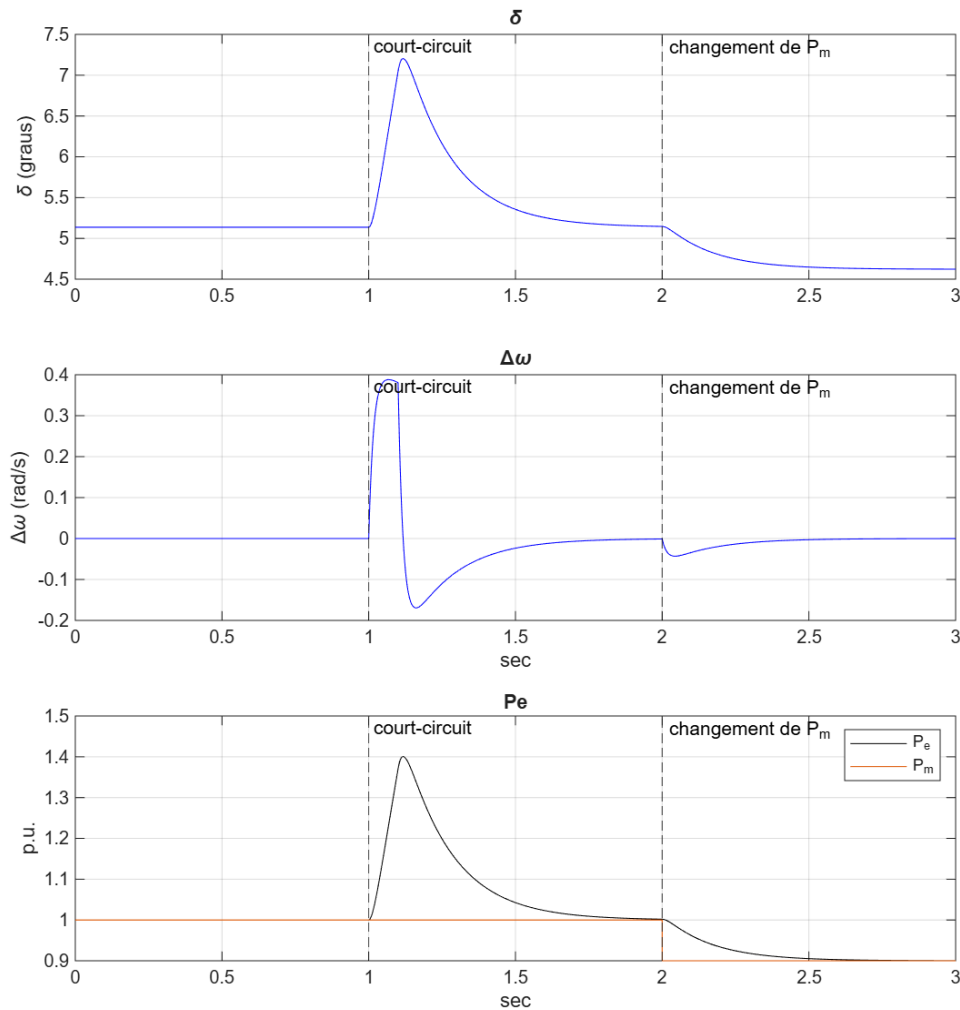


FIGURE 1 – Angle, variation de vitesse et puissances dans le modèle d'ordre 2.

On observe le comportement attendu pour toutes les variables observées. L'angle de charge de la machine a tendance à augmenter lors d'un court-circuit (puissance d'accélération positive, c'est-à-dire une puissance mécanique supérieure à la puissance électrique). À l'inverse, si la puissance mécanique chute, la puissance d'accélération devient négative et l'angle a tendance à se réduire. De la même façon, si l'on a une puissance d'accélération positive (défaut), la vitesse augmente, ce qui entraîne une variation de vitesse positive. Dans l'autre cas, avec une puissance mécanique plus faible, la machine a tendance à ralentir (variation de vitesse négative). Ce comportement est exactement celui observé dans la simulation.

L'autre simulation présentée dans le rapport concerne le modèle d'ordre 6, qui contient tous les éléments présents dans les autres modèles. Son implémentation est montrée dans l'Annexe A. Les Fig. 2, Fig. 3 et Fig. 4 montrent les graphiques obtenus. On peut déjà constater des différences, par exemple le volume d'informations ajoutées par rapport au modèle précédent. Pendant la simulation, on observe un défaut à 1,5 seconde.

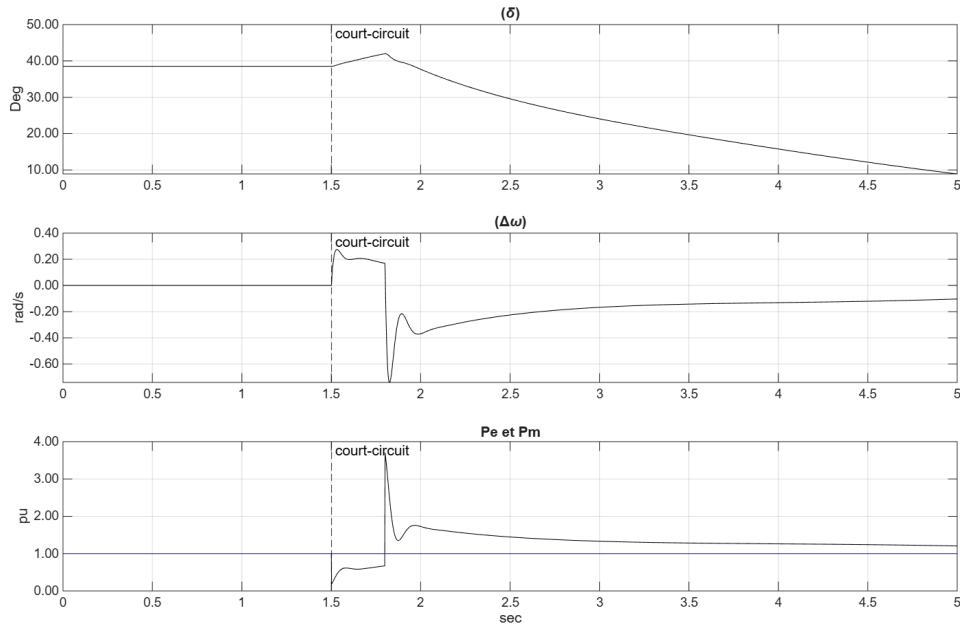


FIGURE 2 – L'angle, variation de vitesse et puissances de la machine.

Dans une première analyse, on observe l'instabilité angulaire de la machine après le défaut. Ce comportement est cohérent avec celui de la puissance électrique observé : celle-ci reste supérieure à la puissance mécanique après le défaut. Selon les Eq. 2 et Eq. 1, on s'attend à une diminution de l'angle et de la variation de vitesse.

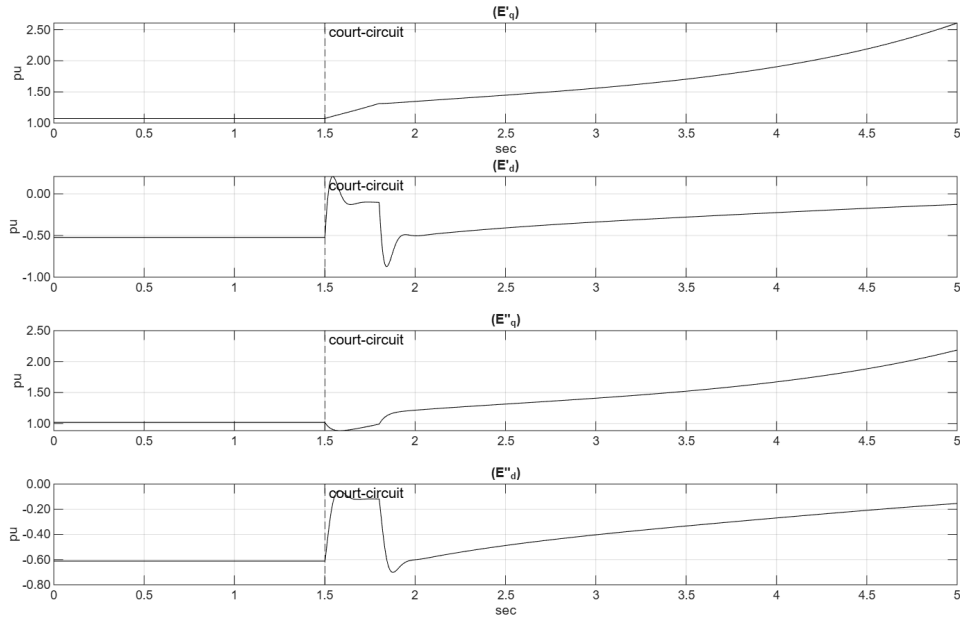


FIGURE 3 – f.e.m. subtransitoires et transitoires.

Cette image montre le comportement (instable après le défaut) des f.e.m. subtransitoires et transitoires de la machine. Notamment, en cas de défaut, il est nécessaire d'ajouter des régulateurs pour la machine, par exemple un régulateur de tension (*automatic voltage regulator* - AVR) et un régulateur de vitesse (*speed governor*). Pour contrôler la machine dans la simulation, on peut intégrer le calcul de la tension de champ E_{fd} (voir l'annexe), mais cela représente un régulateur de tension instantané qui n'existe pas en réalité.

Finalement, la dernière image montre les flux des axes d, q et total. On observe également l'instabilité après le défaut. Ce résultat montre la capacité du modèle à capturer le défaut dans la courbe des flux, notamment dans le flux de l'axe q.

On peut conclure que le principal avantage du modèle d'ordre 6 par rapport au modèle d'ordre 2 est la quantité d'informations fournies, comme le comportement du flux de la machine.

L'effet de la saturation magnétique est expliqué dans le livre mais n'a pas été intégré à la simulation. Le but des prochaines études pourra être de l'ajouter au modèle choisi pour visualiser avec plus de fidélité la variation des réactances des axes d et q, ainsi que des autres grandeurs observées.

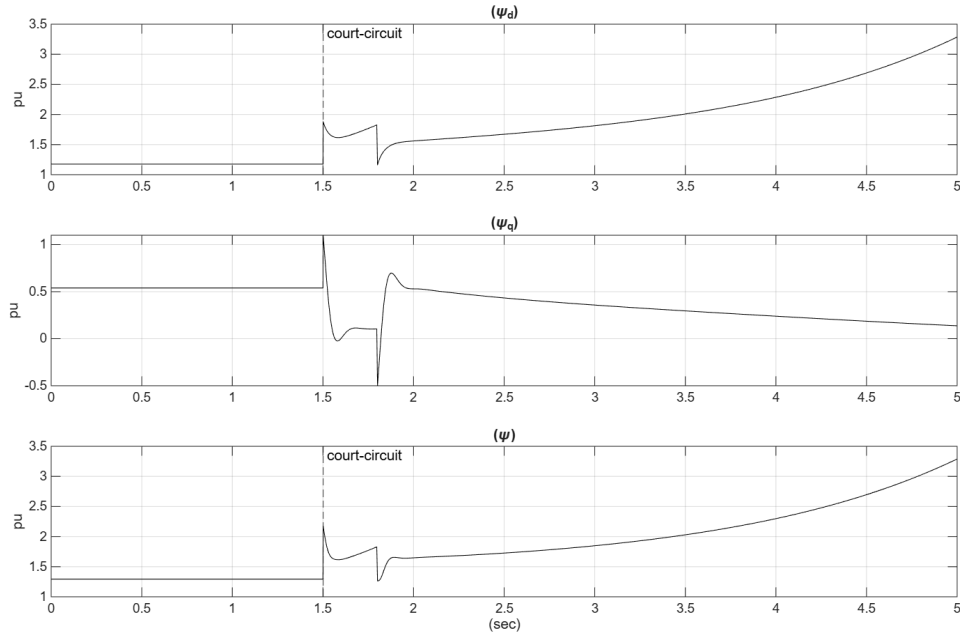


FIGURE 4 – Les flux de la machine (axes d, q et total).

1.2 D'autres modèles de machine synchrone

Une recherche a été réalisée dans le but de trouver d'autres modèles utilisés pour les machines synchrones. Cependant, le nombre de travaux académiques proposant un modèle d'hydro-générateur dans le cadre des jumeaux numériques est restreint.

En se basant sur l'hypothèse que, pour les machines synchrones à pôles saillants de grande puissance, on peut considérer un circuit magnétique linéaire sur l'axe q et une saturation sur l'axe d, [2] propose un modèle décrivant la machine avec l'Eq. 12 en utilisant les flux et courants (modèle de courant donné par [3]).

$$A \frac{dX}{dt} + BX = [u] \quad (12)$$

L'étude présente dans ses résultats le tracé de la variation des inductances L_{md} et L_{mdt} (axe d, axe avec saturation). Ce modèle peut fournir des informations pertinentes sur l'influence de la saturation dans les réactances de la machine, ce qui peut être important pour la détection de défauts.

D'autres approches concernant les machines synchrones à aimants permanents ont été révisées dans le cadre des jumeaux numériques. Notamment, plusieurs travaux [4, 5, 6] proposent des jumeaux numériques basés sur le modèle physique, c'est-à-dire une analyse par éléments finis (*finite element analysis* - FEA), à l'aide d'un entraînement machine pour apprendre le modèle et effectuer des estimations des états de la machine, ainsi que pour détecter des conditions de défaut. [7] propose un modèle hybride qui utilise la FEA pour fournir des données limitées à deux réseaux qui apprennent les dynamiques électromagnétiques et structurelles.

Dans une autre approche, [8] propose l'utilisation de techniques de réduction de dimensionnalité et de tables de correspondance (*look-up tables*) pour augmenter la vitesse de simulation.

Finalement, [9] utilise les modèles physiques (modèle d'ordre 3) pour un moteur à induction comme base d'un filtre de Kalman étendu (*Extended Kalman Filter* - EKF).

Des recherches plus approfondies doivent être effectuées pour trouver des modèles spécifiques de machines synchrones à pôles saillants ou d'hydro-générateurs.

2 Jumeaux Numériques

Après les révisions, recherches et simulations de modèles pour les machines synchrones, d'autres recherches ont été menées sur les jumeaux numériques afin de trouver différentes topologies. Dans les mêmes références citées précédemment, il a été possible d'identifier deux propositions principales.

La première repose sur l'utilisation d'un modèle physique ; plusieurs usages de la FEA permettent de collecter des données sur les variables d'état désirées et d'entraîner des réseaux pour prédire les états de la machine et identifier des événements (comme les court-circuits). Par exemple, [5] utilise la méthode des forêts aléatoires améliorée (*Improved Sparrow Search Algorithm Optimized Random Forest* - ISSA-RF) pour identifier différents types de défauts dans une machine synchrone à aimants permanents.

La deuxième approche se base sur les modèles mathématiques d'observateurs. Par exemple, [9] utilise l'EKF pour estimer la vitesse d'un moteur à induction sans capteur de vitesse ; il utilise les courants de sortie du générateur pour

corriger l'estimation du modèle. En utilisant ce travail comme base, une implémentation simple de test a été réalisée sous MATLAB (avec l'objet natif du filtre) et est présentée dans la section suivante.

2.1 EKF pour les modèles d'ordre 2 et d'ordre 6

Pour tester l'utilisation de l'EKF sur les modèles mentionnés, les équations de base ont été utilisées pour simuler une machine réelle et son fonctionnement véritable. De cette façon, la puissance électrique et la variation de vitesse (pour le modèle d'ordre 2), ainsi que les courants (pour le modèle d'ordre 6), ont été extraits de la simulation en ajoutant du bruit pour simuler les mesures du filtre. Pour l'utilisation du filtre, l'objet `extendedKalmanFilter()` de MATLAB a été employé. La fonction de transition d'état, c'est-à-dire les équations différentielles décrivant la machine, ainsi que les fonctions de mesure (équations des grandeurs mesurées en fonction des variables d'état), constituent les entrées de l'objet.

Le filtre est utilisé pour des modèles décrits par des équations non linéaires; il utilise les jacobiens des fonctions de transition d'état et de mesure pour évaluer l'évolution de l'état actuel vers l'état suivant. En résumé, il utilise les jacobiens pour linéariser les équations non linéaires. Notamment, les jacobiens sont nécessaires; dans le cas où le jacobien analytique n'est pas connu, MATLAB fournit une fonction de différences finies pour les calculer. Cependant, le coût numérique peut augmenter et le filtre peut perdre en précision. Ainsi, les équations étant connues, les jacobiens ont été calculés et fournis directement au filtre. La Fig. 5 montre les mesures utilisées pour le ekf du modèle d'ordre 2.

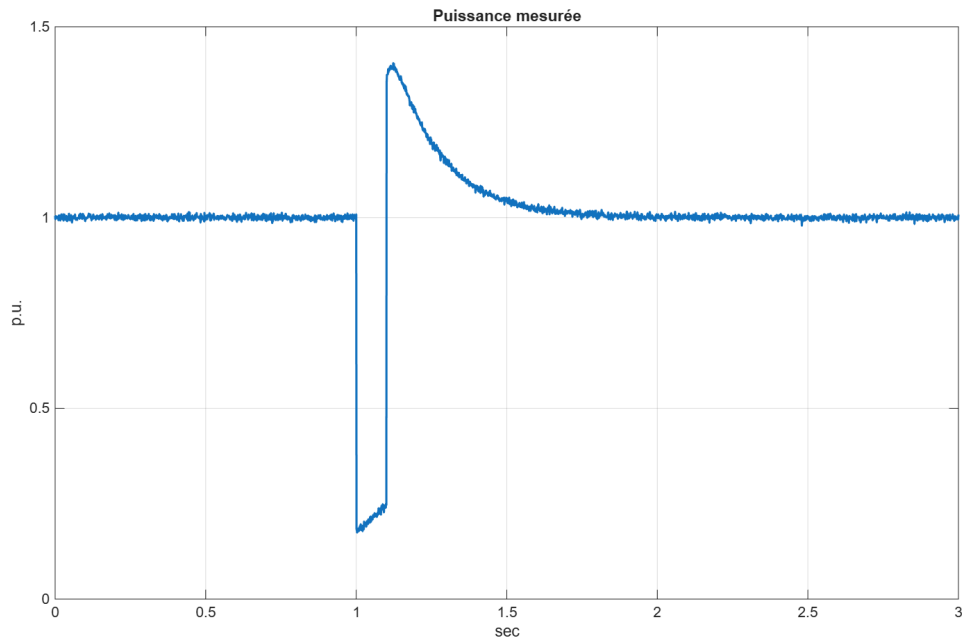


FIGURE 5 – Mesures réalisées de la puissance électrique.

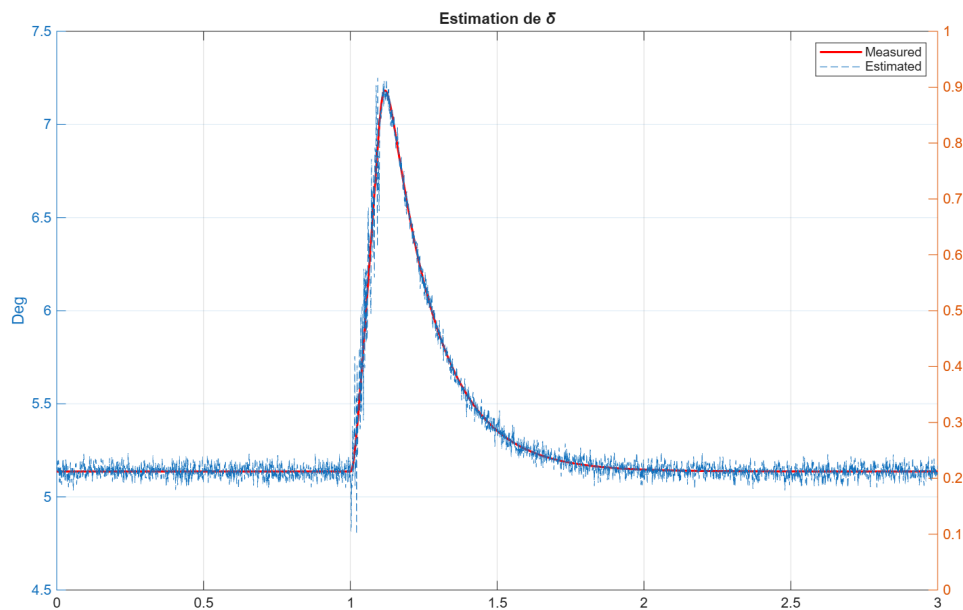


FIGURE 6 – Estimation de l'angle δ .

L'estimation de l'angle est correctement réalisée à partir de la mesure de la puissance électrique ; l'estimation est présentée dans la Fig. 6. L'implémentation complète du filtre pour le modèle d'ordre 2 est déposée dans l'Annexe B.

Enfin, l'implémentation de l'EKF pour le modèle d'ordre 6 a été effectuée. Cependant, il est nécessaire d'augmenter le nombre de mesures du filtre pour obtenir plus de précision. Ainsi, la puissance électrique et les courants sont utilisés pour estimer les états du système sur la base des équations fournies dans la première section. La Fig. 7 montre les mesures simulées pour le filtre.

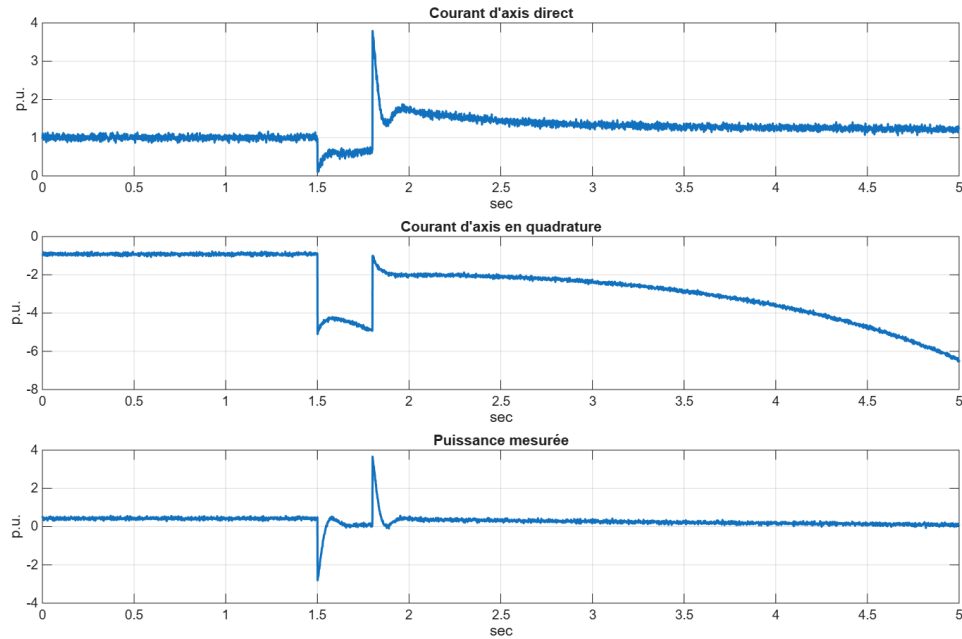


FIGURE 7 – Mesures du filtre.

Les figures suivantes (Fig. 8, Fig. 9 et Fig. 10) présentent les résultats du filtre. On peut visualiser la convergence du filtre pour tous les états. Cependant, on remarque beaucoup plus de bruit dans l'estimation de l'angle des f.e.m. subtransitoires. Cela peut être résolu en modifiant les paramètres du filtre, comme la covariance du bruit de processus (matrice Q dans le code) ou du bruit de mesure (matrice R). Par ailleurs, la précision du filtre est directement liée au niveau de bruit de la mesure.

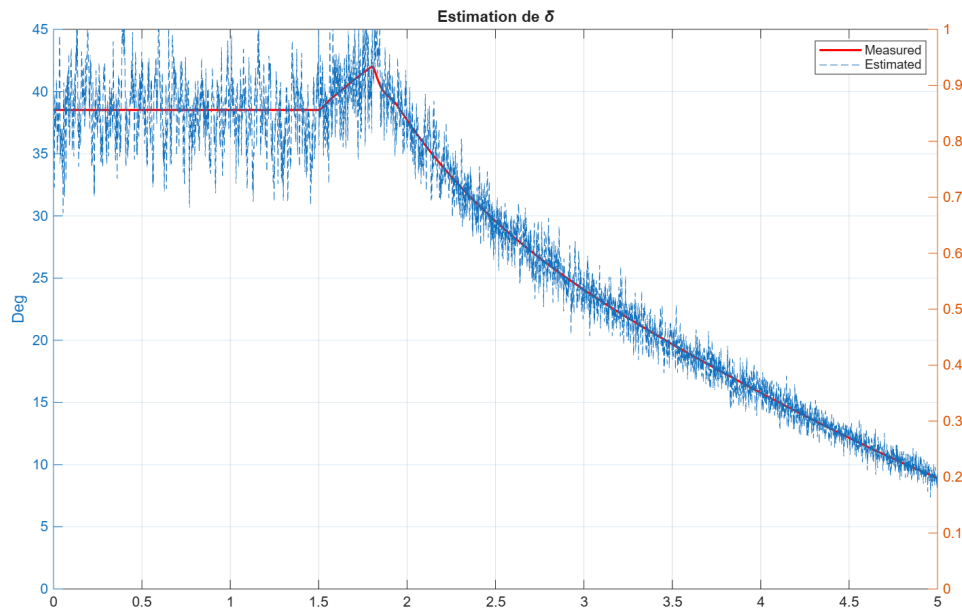


FIGURE 8 – Estimation de l'angle.

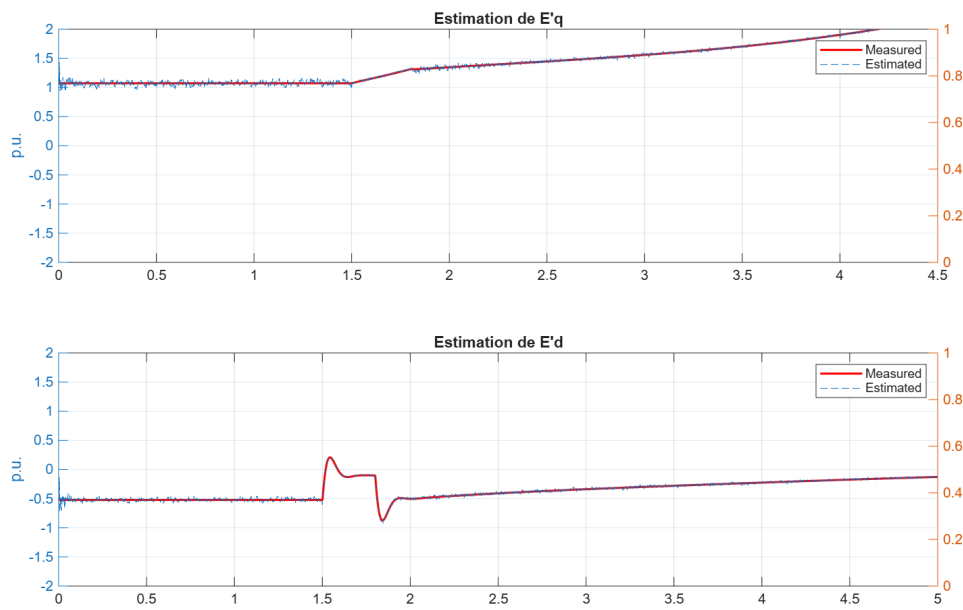


FIGURE 9 – Estimation des f.e.m. transitoires.

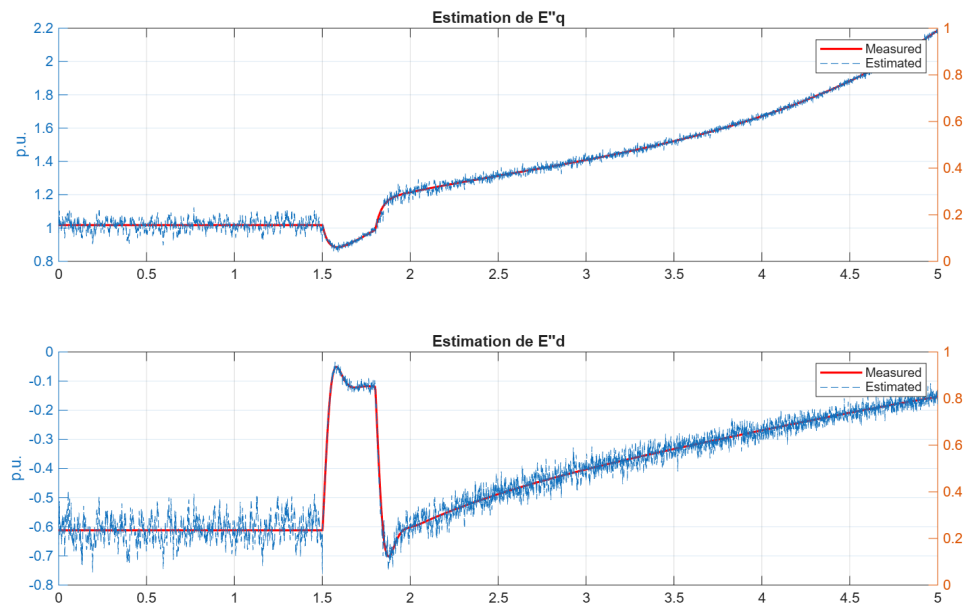


FIGURE 10 – Estimation des f.e.m. subtransitoires.

Les états présentant le plus de bruit sont ceux directement liés à la puissance électrique qui, selon la Fig. 7, est plus bruitée que les courants.

Annexe A

Ci-dessous les paramètres de la machine.

```

1      %% Parametres de la machine
2      H      = 5.0;
3      D      = 2;
4      ws     = 2*pi*50;
5      M      = 2*H/ws;
6      P      = 1;
7      Q      = 0.5;
8
9      Xd     = 1.6;
10     Xd_t   = 0.23;
11     Xd_tt  = 0.1705;
12
13     Xq     = 1.6;

```



```

14      Xq_t  = 0.38;
15      Xq_tt = 0.1703;
16
17      Td0_t = 6;
18      Td0_tt = 0.05;
19
20      Tq0_t = 0.1;
21      Tq0_tt = 0.06;
22
23      Vg    = 1.1;

```

Le code ci-dessous montre l'implémentation du modèle d'ordre 2, ainsi que les équations pour trouver les conditions initiales.

```

1      %% Conditions de depart
2      Pm = 1;
3      delta0 = asin((Pm*Xd_t)/(E_t*Vg));
4      Pe = (E_t*Vg/Xd_t)*sin(delta0);
5      dw0 = 0;
6      x0 = [delta0; dw0];
7
8      %% Description du modele
9      function dxdt = model_ordre2(~, x, V, Xd_t, Pm, E_t, D, M)
10         delta = x(1);
11         dw = x(2);
12
13         ddelta_dt = dw;
14         ddw_dt = (Pm - (E_t*V/Xd_t)*sin(delta) - D*dw)/M;
15
16         dxdt = [ddelta_dt; ddw_dt];
17     end

```

Il en va de même pour le modèle d'ordre 6.

```

1      %% Calcul de la puissance electrique
2      calc_Pe = @(Ed_tt, Id, Eq_tt, Iq, Xd_tt, Xq_tt) (Ed_tt*Id + Eq_tt*Iq + (Xd_tt -
3      Xq_tt)*Id*Iq);
4
5      %% Conditions de depart
6      dw0 = 0;
7      I0 = (P-1i*Q)/Vg;
8      Eq0 = Vg + 1i*Xd*I0;
9      delta0 = angle(Eq0);
10     Id0 = -abs(I0)*sin(-angle(I0) + delta0);
11     Iq0 = abs(I0)*cos(-angle(I0) + delta0);
12     Vgd = -abs(Vg)*sin(delta0);
13     Vgq = abs(Vg)*cos(delta0);
14
15     Ed0_t = Vgd + Xq_t*Iq0;
16     Eq0_t = Vgq - Xd_t*Id0;
17
18     Ed0_tt = Vgd + Xq_tt*Iq0;
19     Eq0_tt = Vgq - Xd_tt*Id0;
20
21     Pe0 = calc_Pe(Ed0_tt, Id0, Eq0_tt, Iq0, Xd_tt, Xq_tt);
22     Pm = Pe0;
23     Ef = Eq0_t + (Xd - Xd_t)*Id0;
24
25     x0 = [delta0; dw0; Eq0_t; Ed0_t; Ed0_tt; Eq0_tt];
26
27     %% Description du modele
28     function dxdt = model_ordre6(t, x, V, Xd, Xd_t, Xd_tt, Xq, Xq_t, Xq_tt, M, D, Pm,
29     Td0_t, Tq0_t, Td0_tt, Tq0_tt)
30         delta = x(1);
31         dw = x(2);
32         Eq_t = x(3);
33         Ed_t = x(4);
34         Ed_tt = x(5);
35         Eq_tt = x(6);

```

```

35 Id = (V * cos(delta) - Eq_tt) / Xd_tt;
36 Iq = (Ed_tt + V * sin(delta)) / Xq_tt;
37
38 Pe = (Ed_tt*Id + Eq_tt*Iq + (Xd_tt - Xq_tt)*Id*Iq);
39 % Efd = Eq_t + (Xd - Xd_t)*Id % Utilise juste pour stabiliser le modele,
40 % il represente un regulateur
41 % instantane qui n'existe pas...
42 ddelta_dt = dw;
43 ddw_dt = (Pm - Pe - D*dw) / M;
44 dEq_t_dt = (Efd - Eq_t - (Xd - Xd_t)*Id) / Td0_t;
45 dEd_t_dt = (-Ed_t - (Xq - Xq_t)*Iq) / Tq0_t;
46 dEq_tt_dt = (Eq_t - Eq_tt + (Xd_t - Xd_tt)*Id) / Td0_tt;
47 dEd_tt_dt = (Ed_t - Ed_tt - (Xq_t - Xq_tt)*Iq) / Tq0_tt;
48
49 dxdt = [ddelta_dt; ddw_dt; dEq_t_dt; dEd_t_dt; dEd_tt_dt; dEq_tt_dt];
50 end

```

```

1 [t1, x1] = ode45(@(t, x) model_ordre6(t, x, Vg, Xd, Xd_t, Xd_tt, Xq, Xq_t, Xq_tt,
M, D, Pm, Td0_t, Tq0_t, Td0_tt, Tq0_tt), [0 t_c], x0);

```

Annexe B

Implémentation de l'EKF pour le modèle d'ordre 2.

```

1 %% Temps de simulation et pour le filtre EKF
2 dt = 1e-3;
3 tpf = 0:dt:1;
4 tf = (1+dt):dt:1.1; % cc
5 tff = (1.1+dt):dt:3;
6
7 %% Fonction d'etat
8 function f = StateFcn(x, V, Xd_t, Pm, E_t, D, M, dt)
9 delta = x(1);
10 dw = x(2);
11
12 ddelta_dt = dw;
13 ddw_dt = (Pm - (E_t*V/Xd_t)*sin(delta) - D*dw)/M;
14
15 f = [ddelta_dt*dt + delta; ddw_dt*dt + dw];
16 end
17
18 %% Fonction de mesure
19 function h = MeasurementFcn(x, V, Xd_t, E_t)
20 delta = x(1);
21 dw = x(2);
22
23 h = [(E_t*V/Xd_t)*sin(delta); dw];
24 end
25
26 %% Jacobien de la fonction d'etat
27 function f_j = StateFcnJacobian(x, V, Xd_t, E_t, D, M, dt)
28 delta = x(1);
29
30 f_j = [1, dt; -(E_t*V/Xd_t)*cos(delta)*dt/M 1-D*dt/M];
31 end
32
33 %% Jacobien de la fonction de mesure
34 function h_j = MeasurementFcnJacobian(x, V, Xd_t, E_t)
35 delta = x(1);
36
37 h_j = [(E_t*V/Xd_t)*cos(delta) 0; 0 1];
38 end
39
40 %% Creation du EKF et simulation des mesures
41 obj = extendedKalmanFilter(@(x)StateFcn(x, Vg, Xd_t, Pm, E_t, D, M, dt), @(x)
MeasurementFcn(x, Vg, Xd_t, E_t), x0, ...
'MeasurementNoise', 1e-3*eye(2), 'ProcessNoise', 1e-2*eye(2), ...

```

```

43         'StateTransitionJacobianFcn', @(x)StateFcnJacobian(x, Vg, Xd_t, E_t, D, M, dt)
44         , 'MeasurementJacobianFcn', @(x)MeasurementFcnJacobian(x, Vg, Xd_t, E_t));
45
46 %% Simulation des mesures pour le filtre
47 V = [ones(size(t1))*Vg; ones(size(t2))*0.2; ones(size(t3))*Vg];
48 measures_ = [(E_t*V/Xd_t).*sin(x(:,1)), x(:,2)];
49 measures = measures_ + 0.005*randn(length(measures_), 2);
50
51 %% Estimation avec EKF
52
53 Vf = [ones(size(t1))*Vg; ones(size(t2))*0.2; ones(size(t3))*Vg] + 0.005*randn(
54     length(measures_), 2) ;
55
56 residBuf = zeros(length(t),2);
57 xcorBuf = zeros(length(t),2);
58 xpredBuf = zeros(length(t),2);
59
60 for k = 1:length(measures(:,1))
61
62     obj.StateTransitionFcn=@(x)StateFcn(x, Vf(k), Xd_t, Pm, E_t, D, M, dt);
63     obj.MeasurementFcn=@(x) MeasurementFcn(x, Vf(k), Xd_t, E_t);
64     obj.StateTransitionJacobianFcn=@(x)StateFcnJacobian(x, Vf(k), Xd_t, E_t, D, M,
65         dt);
66     obj.MeasurementJacobianFcn=@(x)MeasurementFcnJacobian(x, Vf(k), Xd_t, E_t);
67
68     [Residual,ResidualCovariance] = residual(obj,measures(k, :));
69     [CorrectedState,CorrectedStateCovariance] = correct(obj,measures(k, :));
70     [PredictedState,PredictedStateCovariance] = predict(obj);
71
72     residBuf(k,:) = Residual;
73     xcorBuf(k,:) = CorrectedState';
74     xpredBuf(k,:) = PredictedState';
75
76 end

```

Annexe C

Implémentation de l'EKF pour le modèle d'ordre 6.

```

1  %% Temps de simulation et du filtre EKF
2  dt = 1e-3;
3  t_c = 0:dt:1.5;
4  t_cc = 1.5+dt:dt:1.8;
5  t_f = 1.8+dt:dt:5;
6
7  %% Fonction d'etat
8  function f = StateFcn(x, V, Efd, Xd, Xd_t, Xd_tt, Xq, Xq_t, Xq_tt, M, D, Pm, Td0_t
9      , Tq0_t, Td0_tt, Tq0_tt, dt)
10
11  delta = x(1);
12  dw = x(2);
13  Eq_t = x(3);
14  Ed_t = x(4);
15  Eq_tt = x(5);
16  Ed_tt = x(6);
17
18  Id = (V * cos(delta) - Eq_tt) / Xd_tt;
19  Iq = (Ed_tt + V * sin(delta)) / Xq_tt;
20
21  Pe = (Ed_tt*Id + Eq_tt*Iq + (Xd_tt - Xq_tt)*Id*Iq);
22
23  ddelta_dt = dw;
24  ddw_dt = (Pm - Pe - D*dw) / M;
25  dEq_t_dt = (Efd - Eq_t - (Xd - Xd_t)*Id) / Td0_t;
26  dEd_t_dt = (-Ed_t - (Xq - Xq_t)*Iq) / Tq0_t;
27  dEq_tt_dt = (Eq_t - Eq_tt + (Xd_t - Xd_tt)*Id) / Td0_tt;
28  dEd_tt_dt = (Ed_t - Ed_tt - (Xq_t - Xq_tt)*Iq) / Tq0_tt;

```

```

28 f = [ddelta_dt*dt + delta; ddw_dt*dt + dw; dEq_t_dt*dt + Eq_t; dEd_t_dt*dt + Ed_t;
29      dEq_tt_dt*dt + Eq_tt; dEd_tt_dt*dt + Ed_tt];
30
31 end
32
33 %% Fonction de mesure
34 function h = MeasurementFcn(x, V, Xd_tt, Xq_tt)
35     delta = x(1);
36     dw     = x(2);
37     Eq_tt  = x(5);
38     Ed_tt  = x(6);
39
40     Id = (V * cos(delta) - Eq_tt) / Xd_tt;
41     Iq = (Ed_tt + V * sin(delta)) / Xq_tt;
42     Pe = Ed_tt*Id + Eq_tt*Iq + (Xd_tt - Xq_tt)*Id*Iq;
43
44     h = [Pe; dw; Id; Iq];
45 end
46
47 %% Jacobien de la fonction d'etat
48 function f_j = StateFcnJacobian(x, V, Xd, Xd_t, Xd_tt, Xq, Xq_t, Xq_tt, M, D,
49     Td0_t, Tq0_t, Td0_tt, Tq0_tt, dt)
50     delta = x(1);
51     Eq_tt = x(5);
52     Ed_tt = x(6);
53
54     Id = (V * cos(delta) - Eq_tt) / Xd_tt;
55     Iq = (Ed_tt + V * sin(delta)) / Xq_tt;
56
57     dId_ddelta = -V * sin(delta) / Xd_tt;
58     dId_dEq_tt = -1 / Xd_tt;
59     dIq_ddelta = V * cos(delta) / Xq_tt;
60     dIq_dEd_tt = 1 / Xq_tt;
61
62     dPe_ddelta = Ed_tt*dId_ddelta + Eq_tt*dIq_ddelta ...
63         + (Xd_tt-Xq_tt)*(dId_ddelta*Iq + Id*dIq_ddelta);
64
65     dPe_dEq_tt = Ed_tt*dId_dEq_tt + Iq ...
66         + (Xd_tt-Xq_tt)*dId_dEq_tt*Iq;
67
68     dPe_dEd_tt = Id + Eq_tt*dIq_dEd_tt ...
69         + (Xd_tt-Xq_tt)*Id*dIq_dEd_tt;
70
71     J11=1; J12=dt; J13=0; J14=0; J15=0; J16=0;
72
73     J21 = -dPe_ddelta * dt/M;
74     J22 = 1 - D*dt/M;
75     J23 = 0; J24 = 0;
76     J25 = -dPe_dEq_tt * dt/M;
77     J26 = -dPe_dEd_tt * dt/M;
78
79     J31 = -(Xd-Xd_t)*dId_ddelta * dt/Td0_t;
80     J32 = 0;
81     J33 = 1 - dt/Td0_t;
82     J34 = 0;
83     J35 = -(Xd-Xd_t)*dId_dEq_tt * dt/Td0_t;
84     J36 = 0;
85
86     J41 = -(Xq-Xq_t)*dIq_ddelta * dt/Tq0_t;
87     J42 = 0; J43 = 0;
88     J44 = 1 - dt/Tq0_t;
89     J45 = 0;
90     J46 = -(Xq-Xq_t)*dIq_dEd_tt * dt/Tq0_t;
91
92     J51 = (Xd_t-Xd_tt)*dId_ddelta * dt/Td0_tt;
93     J52 = 0;
94     J53 = dt/Td0_tt;
95     J54 = 0;
96     J55 = 1 - dt/Td0_tt + (Xd_t-Xd_tt)*dId_dEq_tt * dt/Td0_tt;
97     J56 = 0;

```

```

95
96 J61 = -(Xq_t-Xq_tt)*dIq_ddelta * dt/Tq0_tt;
97 J62 = 0; J63 = 0;
98 J64 = dt/Tq0_tt;
99 J65 = 0;
100 J66 = 1 - dt/Tq0_tt - (Xq_t-Xq_tt)*dIq_dEd_tt * dt/Tq0_tt;
101
102 f_j = [J11 J12 J13 J14 J15 J16;
103        J21 J22 J23 J24 J25 J26;
104        J31 J32 J33 J34 J35 J36;
105        J41 J42 J43 J44 J45 J46;
106        J51 J52 J53 J54 J55 J56;
107        J61 J62 J63 J64 J65 J66];
108
109 end
110
111 %% Jacobien de la fonction de mesure
112 function h_j = MeasurementFcnJacobian(x, V, Xd_tt, Xq_tt)
113     delta = x(1);
114     Eq_tt = x(5);
115     Ed_tt = x(6);
116
117     Id = (V * cos(delta) - Eq_tt) / Xd_tt;
118     Iq = (Ed_tt + V * sin(delta)) / Xq_tt;
119
120     dId_ddelta = -V * sin(delta) / Xd_tt;
121     dId_dEq_tt = -1 / Xd_tt;
122     dIq_ddelta = V * cos(delta) / Xq_tt;
123     dIq_dEd_tt = 1 / Xq_tt;
124
125     dPe_ddelta = Ed_tt*dId_ddelta + Eq_tt*dIq_ddelta ...
126                 + (Xd_tt-Xq_tt)*(dId_ddelta*Iq + Id*dIq_ddelta);
127     dPe_dEq_tt = Ed_tt*dId_dEq_tt + Iq + (Xd_tt-Xq_tt)*dId_dEq_tt*Iq;
128     dPe_dEd_tt = Id + Eq_tt*dIq_dEd_tt + (Xd_tt-Xq_tt)*Id*dIq_dEd_tt;
129
130     h_j = [ dPe_ddelta,    0,    0,    0,    dPe_dEq_tt,    dPe_dEd_tt ;
131            0,            1,    0,    0,    0,            0 ;
132            dId_ddelta,    0,    0,    0,    dId_dEq_tt,    0 ;
133            dIq_ddelta,    0,    0,    0,    0,            dIq_dEd_tt ];
134
135 end

```

Références

- [1] J. Machowski, Z. Lubosny, J. W. Bialek, and J. R. Bumby, *Power System Dynamics : Stability and Control*, 3rd ed. Chichester, UK : Wiley, 2020.
- [2] T. Dordea, R. Munteanu, and A. Câmpăanu, "Simulation of dynamic operation in salient pole synchronous machines," in *Proceedings of the 2011 International Conference on Electromotion (ELECTROMOTION 2011)*, Istanbul, Turkey, Sept. 2011, pp. 143–148, 8-10 September 2011.
- [3] A. Câmpăanu and M. Stiebler, "Modeling and Simulation of Dynamical Processes in High Power Salient Pole Synchronous Machines," *Revue Roumaine des Sciences Techniques - Série Électrotechnique et Énergétique*, vol. 56, no. 2, pp. 143–152, 2011.
- [4] S. Venkatesan, K. Manickavasagam, N. Tengenai, and S. Vijayalakshmi, "Digital Twin in Electrical Machine Control and Predictive Maintenance : State-of-the-Art and Future Prospects," *Energies*, vol. 14, no. 18, p. 5933, 2021, special Issue : Digital Twin in Electrical Machine Control and Predictive Maintenance.
- [5] Y. Huang, B. Yuan, S. Xu, and T. Han, "Fault Diagnosis of Permanent Magnet Synchronous Motor of Coal Mine Belt Conveyor Based on Digital Twin and ISSA-RF," *Processes*, vol. 10, no. 9, p. 1679, 2022. [Online]. Available : <https://doi.org/10.3390/pr10091679>
- [6] G. F. Lukman and C. Lee, "Towards Digital Twin Modeling and Applications for Permanent Magnet Synchronous Motors," *Energies*, vol. 18, no. 4, p. 956, 2025. [Online]. Available : <https://doi.org/10.3390/en18040956>
- [7] S. Son, H. Lee, D. Jeong, K. h. Sun, and K.-Y. Oh, "Digital twin model of a permanent magnet synchronous motor via a multiphysics-informed deep operator network," *Mechanical Systems and Signal Processing*, vol. 229, p. 112561, 2025.
- [8] S. Venkatesan, K. Manickavasagam, N. Tengenai, and S. Vijayalakshmi, "Digital Twin in Electrical Machine Control and Predictive Maintenance : State-of-the-Art and Future Prospects," *Energies*, vol. 14, no. 18, p. 5933, 2021. [Online]. Available : <https://doi.org/10.3390/en14185933>

- [9] M. Ebadpour, M. Jamshidi, J. Talla, H. Hashemi-Dezaki, and Z. Peroutka, “Digital Twin Model of Electric Drives Empowered by EKF,” *Sensors*, vol. 23, no. 4, p. 2006, 2023. [Online]. Available : <https://doi.org/10.3390/s23042006>